

Unit Tests

Write our own tests to solve the problems we have been trying to solve and make sure the code runs properly.

Use:

```
if __name__ == "__main__":  
    ... main()
```

So that the python file is treated as a library if it ever gets imported

To test certain functions of a library/module, make a separate program (usually "test_filename") to test a specific program.

There is a convention used when if the user wants to test a certain function, the function for testing should be defined:

`def test_functionName():` or `def functionName_test():`

ex. testing a function that squares numbers

```
def test_square():  
    if square(2) != 4:  
        print("LError")  
    if square(3) != 9:  
        print("LError")
```

<prints nothing if "Square" function is running properly>

assert: function that asserts that something is true/false

ex. using assert

```
def test_square():  
    ... assert square(2) == 4  
    ... assert square(3) == 9
```

(assume "Square" function errors)

AssertionError

pytest: third party program to install, that will automate the testing of code, as long as we write the tests.

Unit Testing: formal way of describing testing individual units of program, which are typically functions.

When using pytest, download the same way in terminal window:
"pip install pytest"

but running pytest should also be in terminal.

python Code.py

↑
to run the Code.py file

pytest Code.py

↑
to test the Code.py file

if Pytest fails:

in terminal: `pytest Code.py`

F

[100%]

Failures

function Name

def test_functionName():

> assert functionName(val1) == val2

E assert val3 == val2

E + where [val3] = functionName(val1)

Code.py : 3: AssertionError

1 failed in <time>

if Pytest passes:

[100%]

passed in <time>

Pytest will test an entire program, not individual functions from a program that has been called. This means the user can make multiple functions with "def" to test different categories of unit tests, and retrieve multiple errors to help determine the problem.

Pytest can also run a whole folder of tests (mentioned below).

Pytest itself is a library that can be imported into a file, and has functions within its documentation.

ex. using the "Square" function and importing pytest:

```
import pytest
```

```
def test_sq():
```

```
    with pytest.raises(TypeError):
        Square("cat")
```

```
>>> pytest test_calc.py
[100%]
passed in <time>
```

If the function that is being tested does not return a value, even though it works properly, the pytest will fail.

To run pytest on a folder, the folder of choice must have another python file with the name "`--init--.py`", and does not need to have anything else. This treats the folder as a package; like a module in folder format.

ex. Code `test.py` ← file with functions
mkdir TheTest ← make new directory/folder
Code TheTest/testing ← file that tests file w/ functions
Code `--init--.py` ← package indicator
pytest TheTest ← pytest "tests" the whole folder